

Biomedical Computer Vision

Sharpening & Edge Detection

March 24th, 2026

Virginia Tasso
virginia.tasso@polimi.it



Syllabus:

17.03

- Data dimensionality, image formats
- Image preprocessing

19.03

- Introduction to convolution
- Padding, stride, pooling

24.03

- Convolution-based operation: filtering (denoising, sharpening, edge detection)

27.03

- Introduction to segmentation: thresholding, masking, k-means...
- Post-processing: connected components, erosion, dilation...

31.03

- Introduction to Deep Learning
- Fully Connected Neural Networks, Convolutional Neural Networks

02.04

- NN training and regularization (dropout, weight decay...)

14.04

- Image segmentation with Deep Learning
- UNet

16.04

- CNN and others for other tasks (generation, denoising...)
- Review of the state of the art

Syllabus:

17.03

- Data dimensionality, image formats
- Image preprocessing

19.03

- Introduction to convolution
- Padding, stride, pooling

24.03

- Convolution-based operation: filtering (denoising, sharpening, edge detection)

27.03

- Introduction to segmentation: thresholding, masking, k-means...
- Post-processing: connected components, erosion, dilation...

31.03

- Introduction to Deep Learning
- Fully Connected Neural Networks, Convolutional Neural Networks

02.04

- NN training and regularization (dropout, weight decay...)

14.04

- Image segmentation with Deep Learning
- UNet

16.04

- CNN and others for other tasks (generation, denoising...)
- Review of the state of the art

Image Enhancing - Sharpening

Image sharpening is a process that enhances the definition of edges and fine details within an image, without completely removing low-frequency components. Most of the techniques include some form of high pass filtering. This process makes features like boundaries, textures, and structures appear clearer and more distinct. e.g., in the medical field, it is critical for improving the visibility of anatomical structures.

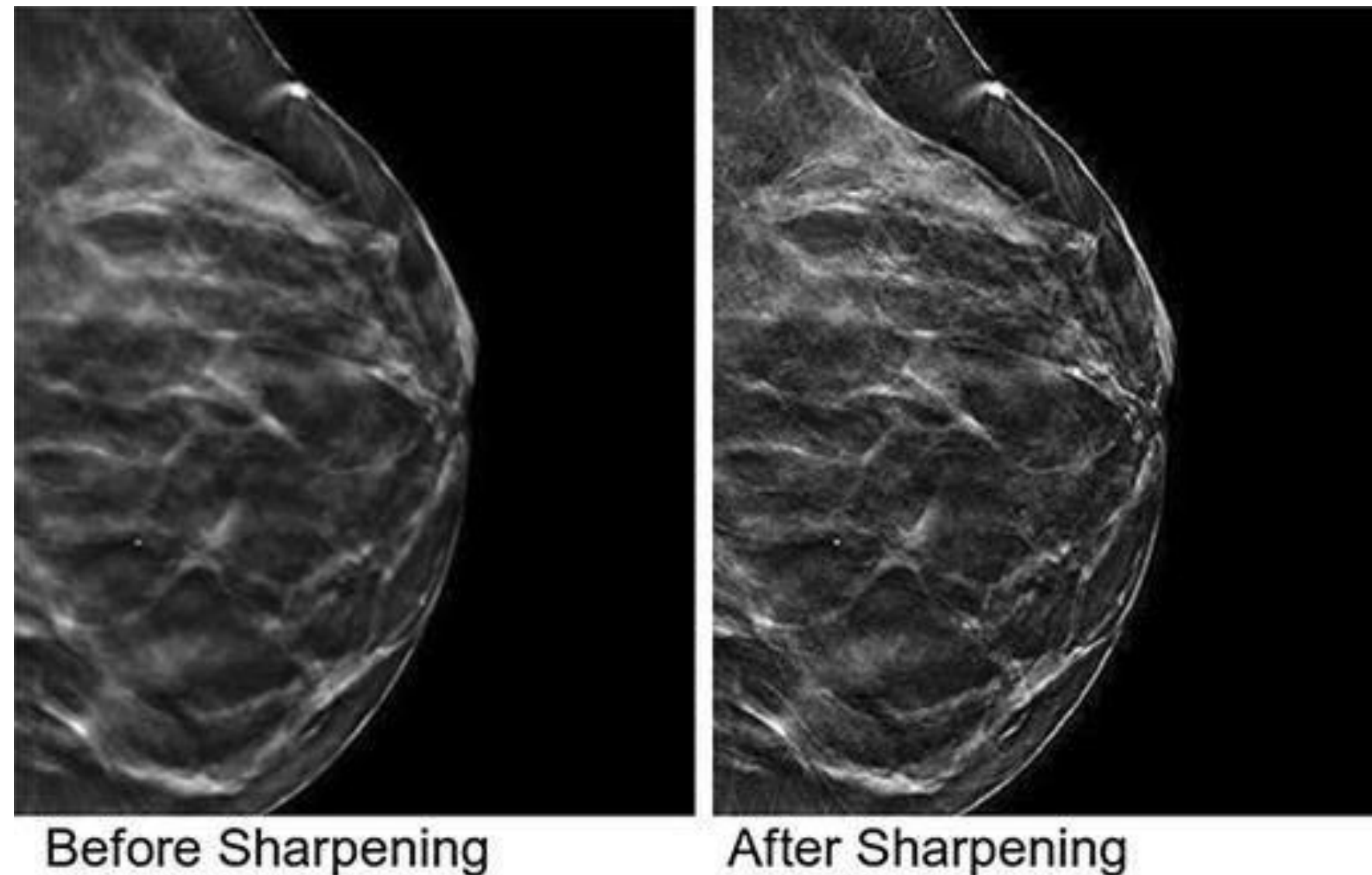
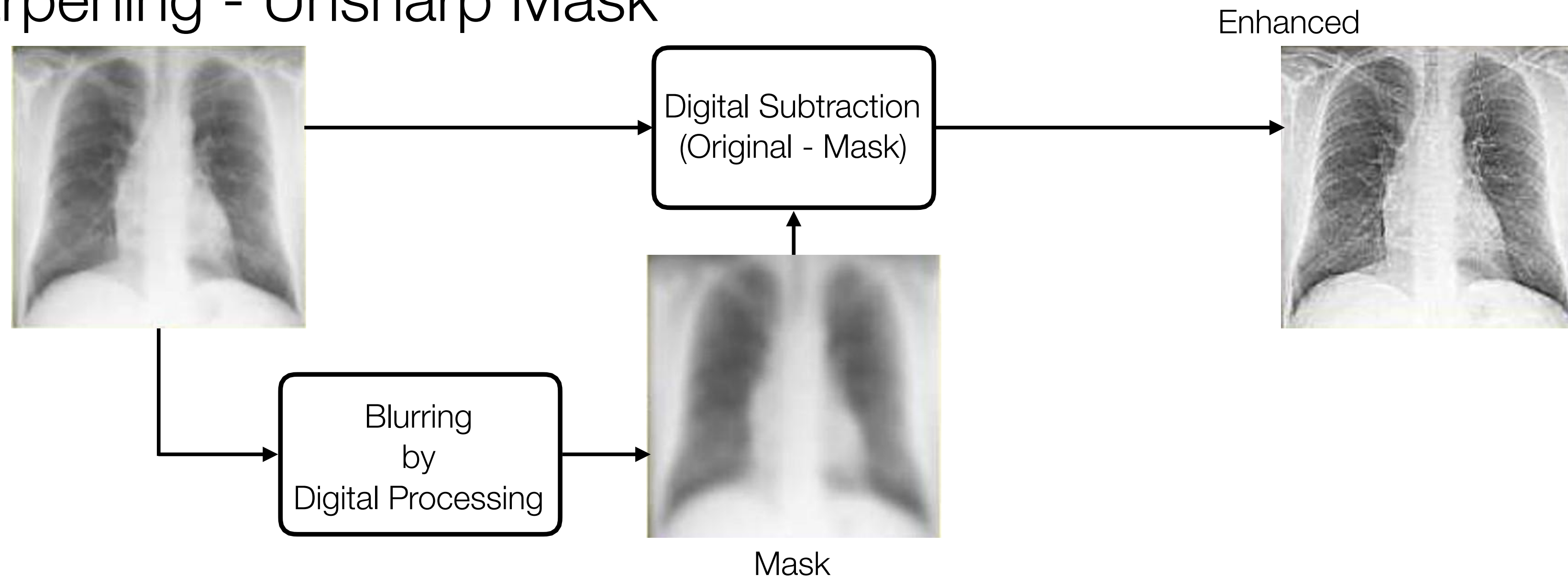


Image Enhancing - Sharpening

Many image sharpening algorithms consist of four general steps:

1. Filtering out the noise
2. Extracting high frequency information (High pass filtering)
3. Combining the high frequency image with the original image to emphasize image detail
4. Maximizing image contrast via histogram manipulation

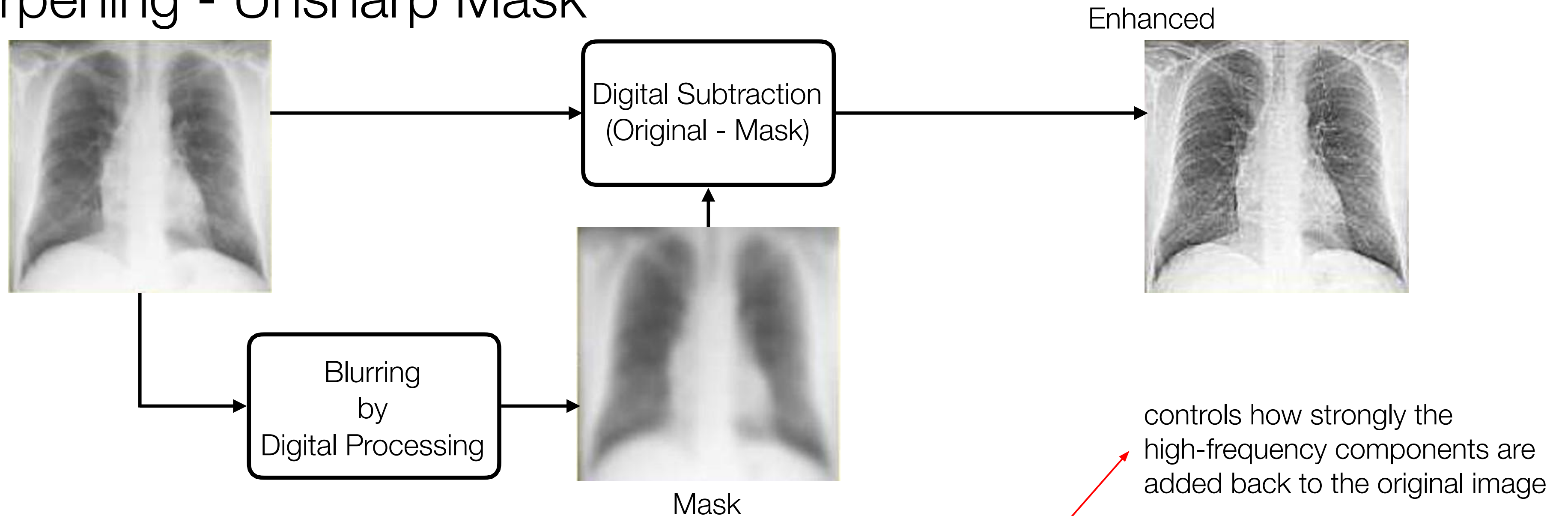
Sharpening - Unsharp Mask



This process works because subtracting a slowly changing edge (the lowpass filtered image) from faster changing edges (in the original), has the visual effect of causing overshoot and undershoot at the edges, which has the effect of emphasizing the edges.

$$\text{Sharpened} = \text{original} + \text{amount} * (\text{original} - \text{blurred})$$

Sharpening - Unsharp Mask



This process works because subtracting a slowly changing edge (the lowpass filtered image) from faster changing edges (in the original), has the visual effect of causing overshoot and undershoot at the edges, which has the effect of emphasizing the edges.

$$\text{Sharpened} = \text{original} + \text{amount} * (\text{original} - \text{blurred})$$

Sharpening - High Boost SpatialFilter

It is a single-kernel implementation designed to do all 3 steps of unsharp masking at the same time. It consists of a 3x3 kernel.

All surrounding pixels have value -1: this part of the kernel capture the blurred version of the image.

The center x determines how much of the original image is retained.

Sharpening - High Boost SpatialFilter

- A value of 8 will result in a high pass filter (the output image will contain only the edges), while larger values will retain more of the original image.
- If values of less than 8 are used for x , the resulting image will appear as a negative of the original.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & x & -1 \\ -1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & x - 8 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Laplacian filter

Scaled identity kernel

$$A = x - 8$$

Sharpening - High Boost SpatialFilter

- A value of 8 will result in a high pass filter (the output image will contain only the edges), while larger values will retain more of the original image.
- If values of less than 8 are used for x , the resulting image will appear as a negative of the original.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & x & -1 \\ -1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & x - 8 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

High-pass filtering.
Retains only the
edges

Scaled identity kernel

$$A = x - 8$$

Sharpening - High Boost SpatialFilter

- A value of 8 will result in a high pass filter (the output image will contain only the edges), while larger values will retain more of the original image.
- If values of less than 8 are used for x , the resulting image will appear as a negative of the original.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & x & -1 \\ -1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & x - 8 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

High-pass filtering.
Retains only the
edges

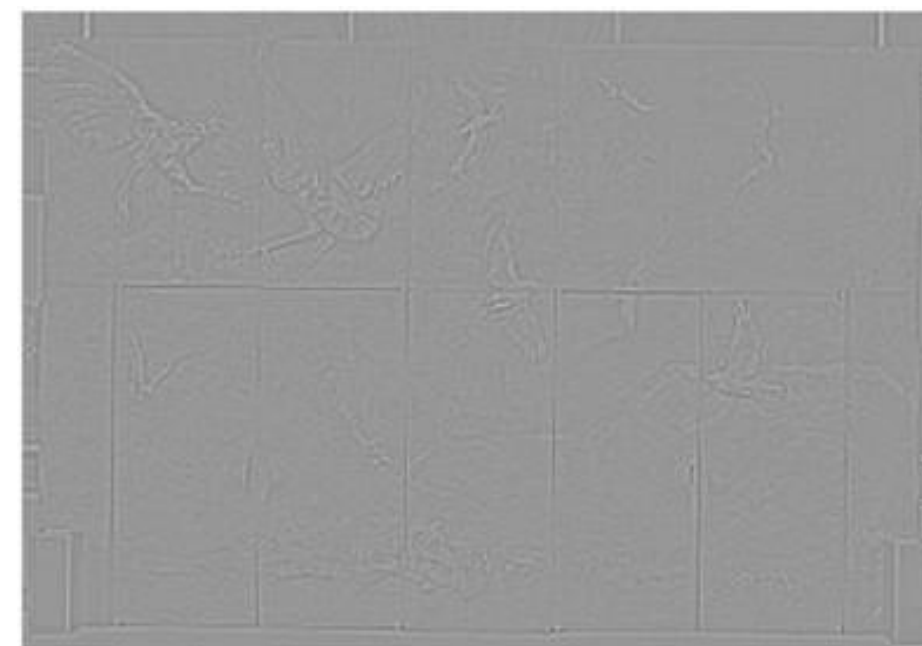
Determine how much
of the original image is
retained

Sharpening - High Boost SpatialFilter

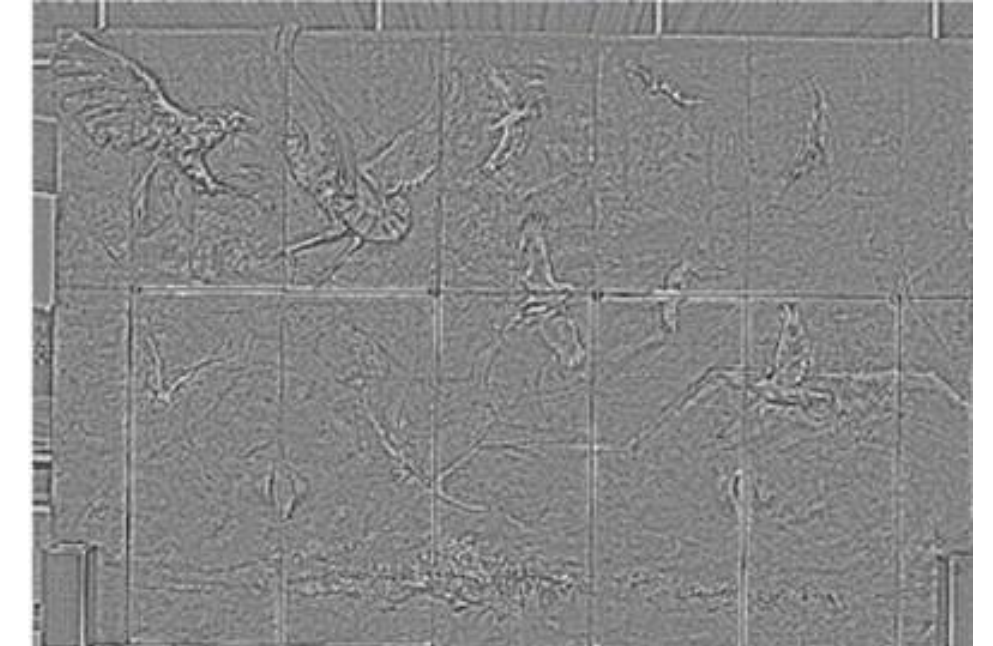
$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & x & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



Original



$x = 8$



$x = 8$ stretched

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & x & -1 \\ -1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

output = edges + 0 * original image

We are completely removing the original image

Sharpening - High Boost SpatialFilter

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & x & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



Original



$x > 8$



$x > 8$ stretched

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & x & -1 \\ -1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} + \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 - 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

output = edges + 1 * original image

We are summing to the edges, the original image, so we obtain a sharpened version

Sharpening - High Boost SpatialFilter

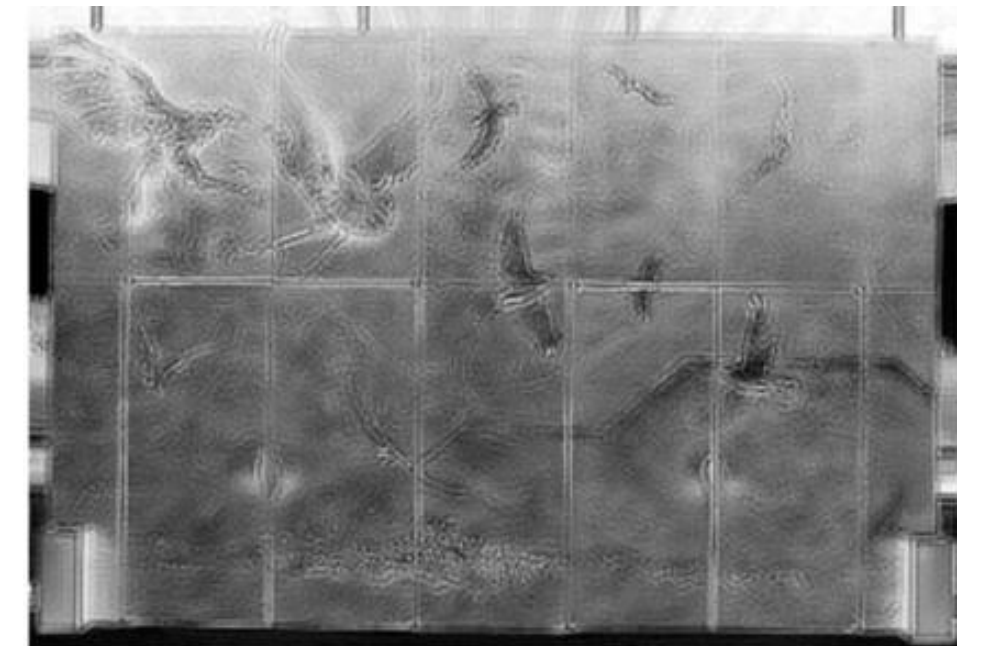
$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & x & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



Original



$x < 8$



$x < 8$ stretched

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & x & -1 \\ -1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & -7 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

output = edges - 1 * original image

We are subtracting the original image from the edges, therefore we obtain an inverted image

Edge Detection

Edge detection is a process for identifying points in an image where brightness changes sharply, which typically correspond to boundaries of objects or regions within the image.

These sharp changes almost always correspond to the boundaries of objects or different regions within the image.

In medical imaging it is helpful to segment and identify anatomical structures, tumors, lesions, or other regions of interest.

Edge Detection

This approach is based on the searching for discontinuity using the first (gradient) and the second (Laplacian) derivatives.

$$\nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

A fundamental property of the gradient vector is that it is oriented towards the direction of the maximal variation of f in the coordinates (x,y) . Such angle is given by:

$$\alpha(x, y) = \tan^{-1} \left(\frac{G_x}{G_y} \right)$$

Edge Detection

The Laplacian is not commonly utilized by itself as edge detector, as it is very sensitive to noise. Moreover, it results in a double contour and it is not able to indicate the edge direction. However, it is utilized together with other edge detectors, to localize potential edges:

$$\nabla^2 f(x, y) = \frac{\partial^2 f(x, y)}{\partial x^2} + \frac{\partial^2 f(x, y)}{\partial y^2}$$

One of the key problems is how to estimate the directional derivatives as digital approximations.

Edge Detection - MostCommon

Sobel

Each filter takes the derivative in one direction and smoothes in the orthogonal direction using a 1D version of a *triangular* filter.

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

Edge Detection - MostCommon

Sobel

Each filter takes the derivative in one direction and smoothes in the orthogonal direction using a 1D version of a *triangular* filter.

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

Prewitt

It is based on the gradient approximation according to Prewitt (max gradient). Each filter takes the derivative in one direction and smoothes in the orthogonal direction using

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

Edge Detection - MostCommon

Sobel

Each filter takes the derivative in one direction and smoothes in the orthogonal direction using a 1D version of a *triangular* filter.

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

Prewitt

Each filter takes the derivative in one direction and smoothes in the orthogonal direction using a uniform filter.

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

Roberts

It is based on the gradient approximation according to Roberts.

-1	0
0	1

0	-1
1	0

Edge Detection - Sobel

Each filter takes the derivative in one direction and smooths in the orthogonal direction using a 1D version of a *triangular* filter.



-1	-2	-1
0	0	0
1	2	1

Derivative
in x



-1	0	1
-2	0	2
-1	0	1

Derivative
in y



Laplacian



Edge Detection - Boxcar Operators

- Edge detector results are not as good when noise is added to the image
- To mitigate noise effects, preprocessing the image with mean spatial filters can be done
- Additionally, the size of the edge detection masks can be extended for noise mitigation
- An example of this method is to extend the Sobel edge mask as follows Called *boxcar operators* and can be extended: 7x7, 9x9 and 11x11 are typical

$$\begin{bmatrix} 1 & 1 & 1 & 0 & -1 & -1 & -1 \\ 1 & 2 & 2 & 0 & -2 & -2 & -1 \\ 1 & 2 & 3 & 0 & -3 & -2 & -1 \\ 1 & 2 & 3 & 0 & -3 & -2 & -1 \\ 1 & 2 & 3 & 0 & -3 & -2 & -1 \\ 1 & 2 & 2 & 0 & -2 & -2 & -1 \\ 1 & 1 & 1 & 0 & -1 & -1 & -1 \end{bmatrix}$$

Edge Detection - Boxcar Operators

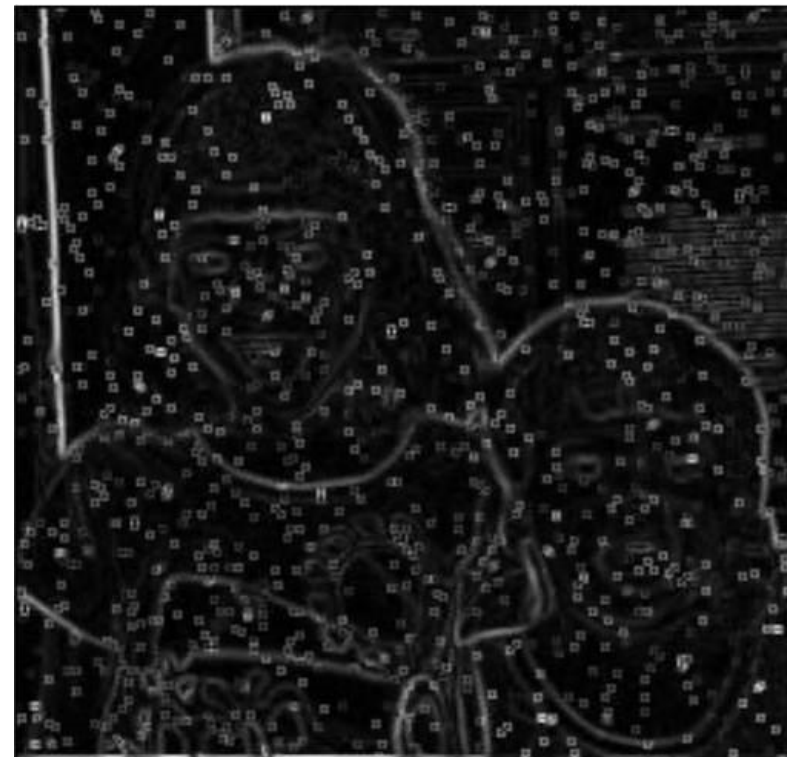
Original



Original + S&P Noise



Sobel

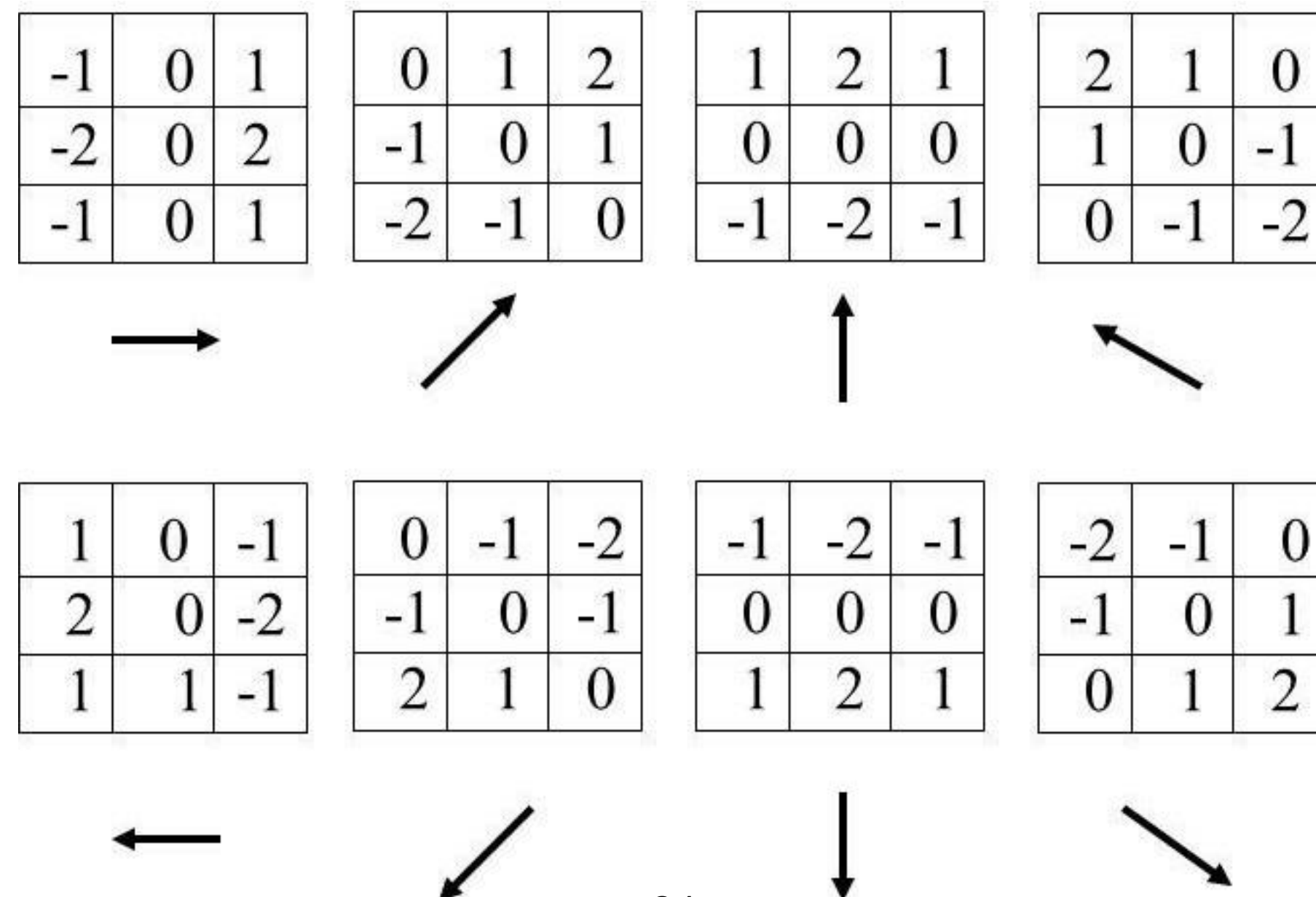


Sobel with Boxcar



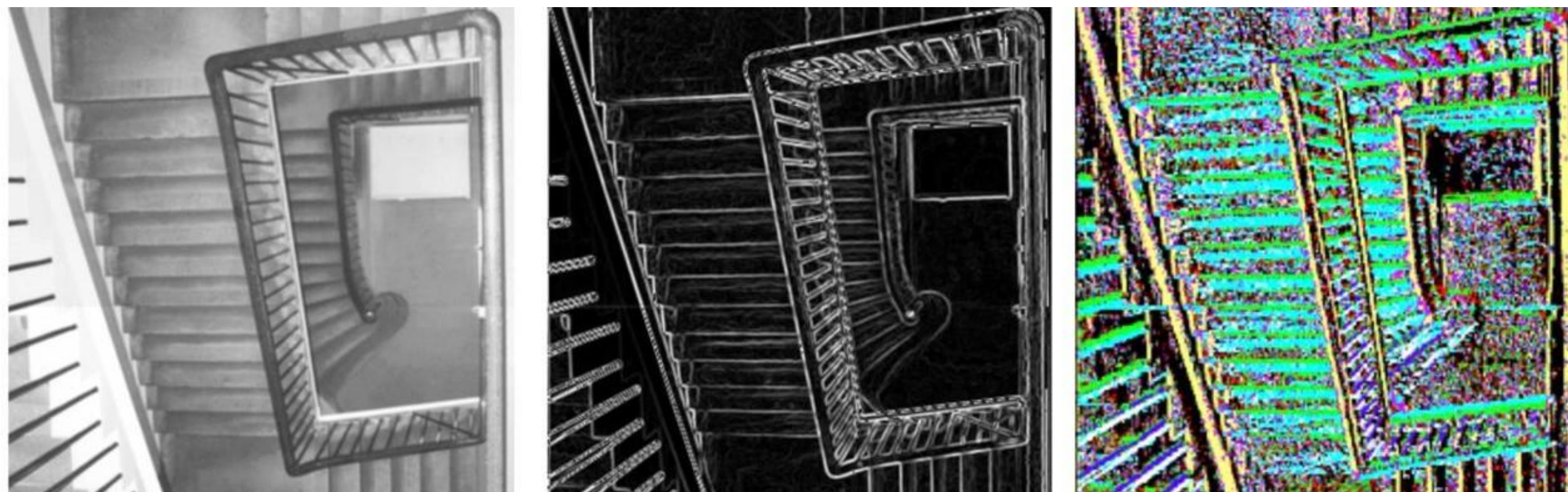
Edge Detection - Compass Mask

The *Kirsch* and *Robinson* edge detection masks are called compass masks since they are defined by taking a single mask and rotating it to the eight major compass orientations: North, Northwest, West, Southwest, South, Southeast, East, and Northeast



Edge Detection - Compass Mask

- The edge magnitude is defined as the maximum value found by the convolution of each of the masks with the image.
- The edge direction is defined by the mask that produces the maximum magnitude.
- In general, any of the edge detection masks can be extended by rotating them in a manner like the compass masks, which allows us to extract explicit information about edges in any direction.



Edge Detection - Laplacian Operator

- The Laplacian masks are rotationally symmetric, which means edges at all orientations contribute to the result (they do not favour any particular direction).
- Applied by selecting one mask and convolving it with the image.
- The sign of the result (positive or negative) from two adjacent pixel locations provides directional information, and tells us which side of the edge is brighter (it provides the polarity).
- If we increase the center coefficient by one it is equivalent to adding the original image to the edge detected image (image sharpening).

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

$$\begin{bmatrix} -2 & 1 & -2 \\ 1 & 4 & 1 \\ -2 & 1 & -2 \end{bmatrix}$$

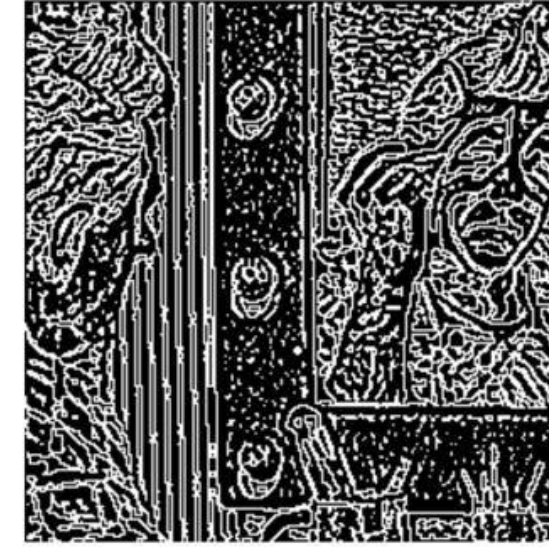
Edge Detection - Laplacian of Gaussian (LoG)

- These edge detectors are considered to be advanced because they are algorithmic in nature
- They are based on the idea that most edge detectors are too sensitive to noise and thus, by blurring the image prior to edge detection, it is possible to mitigate these noise effects.
- An interesting aspect of the LoG operator is that it is believed to closely model biological vision systems

A common 5x5 mask that approximates the combination of the Gaussian and Laplacian into one convolution mask is as follows:

$$\begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ -1 & -2 & 16 & -2 & -1 \\ 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \end{bmatrix}$$

Edge Detection - Laplacian of Gaussian (LoG)



LoG filter
with SD=1
(offset
128)

LoG filter
with SD=2
(offset
128)



LoG filter
with SD=3
(offset
128)



Edge Detection - Canny

Multistage edge detector algorithm and it is one of the most optimal and reliable edge detection methods due to its ability of detecting as many true edges as possible, while minimizing false detections.

- Smooth the image with Gaussian Filter
- Compute the gradient magnitude and orientation using finite difference for partial derivatives (Roberts, Prewitt, Sobel)
- non-maxima suppression: a search is carried out to determine if the gradient magnitude assumes a local maximum in the gradient direction.

Es: if the rounded gradient angle is zero degrees (i.e. north- south direction) the point will be considered to be on the edge if its intensity is greater than the intensities in the west and east directions, otherwise will be suppressed. A set of edge points ("thin edges"), in the form of a binary image, is obtained.

Edge Detection - Canny

- Use hysteresis thresholding (two thresholds – high and low) to detect and link edges.
- We begin by applying a high threshold. This marks out the edges we can be fairly sure are genuine. Starting from these, using the directional information derived earlier, edges can be traced through the image. While tracing an edge, we apply the lower threshold, allowing us to trace faint sections of edges as long as we find a starting point.



Biomedical Computer Vision

November 11th, 2025

Virginia Tasso
virginia.tasso@polimi.it

